

Task 3 : Learning to Rank

Information-retrieval systems used by recruitment platforms, search engines, and recommendation services share a structural property that distinguishes them from standard supervised-learning tasks, their predictions are evaluated within query groups rather than over the entire test set. For a job posting with one hundred applicants, what matters is the relative ordering of those one hundred candidates, not their absolute predicted scores, and not the ordering of candidates across different job postings. This family of problems is known as Learning to Rank and constitutes the algorithmic backbone of systems such as Google Search, LinkedIn Recruiter, Indeed, and Airbnb's listing ranker. In this task, you will design and implement a complete LTR pipeline for a synthetic but realistic tech-recruitment dataset.

Dataset Overview:

The dataset comprises three files:

- jobs.csv: $\approx 5,000$ job postings with 12 attributes.
- candidates.csv: $\approx 50,000$ candidate profiles.
- applications_train.csv / applications_test.csv: The training file contains the label, the test file does not. Submissions are made against applications_test.csv.

The train/test split is temporal, based on application_date:

- Training set: Applications with application_date < 2024-07-01
- Test set: Applications with application_date $\geq$ 2024-07-01

The test set includes:

- Applications for previously-seen jobs (jobs that also appear in training)
- Applications for new jobs (jobs that appear only in test set)

This tests your model's ability to rank both familiar and unfamiliar job contexts. Build features that generalize across job types.

You are provided with two years of recruitment data, TalentMatch. For every (job, candidate) pair in the training set, you are given a graded relevance label *relevance label* $\in \{0. 1. 2. 3. 4\}$ reflecting how well the candidate fits that specific posting. Your objective is to learn a scoring function $f(\text{job}, \text{candidate}) \rightarrow \mathbb{R}$ such that, when candidates are sorted in descending order of f within each job_id, the resulting ranking approximates the hidden ground-truth ranking on a held-out test set.

Each label reflects candidate_job fit:

- Label 0: Not relevant at all (poor match, recommend rejection)
- Label 1: Slightly relevant (some mismatch, marginal fit)
- Label 2: Moderately relevant (reasonable fit, worth reviewing)
- Label 3: Highly relevant (strong match, advance to interview)
- Label 4: Extremely relevant / Best match (ideal candidate, recommend hiring)

Labels are assigned per_job, not globally. A candidate may receive label 4 for one job and label 1 for another job. This is intentional and expected.

Process Overview:

1. **Data Exploration & Understanding:** Exploratory Data Analysis (EDA) is a critical first step. You are expected to thoroughly explore the data, create visualisations, and identify relationships between features and the target. In the in-person session, you must justify your preprocessing, feature engineering, and model choices based on your EDA findings. EDA and visualisations are part of the overall evaluation.
2. **Preprocessing & Data Cleaning:** Build a fully reproducible preprocessing pipeline for the dataset. The pipeline should appropriately handle missing values, categorical variable encoding, and normalization or scaling of numeric features. If target encoding is used, it must be implemented using out-of-fold (OOF) target encoding in order to avoid target leakage. You are free to use any preprocessing strategy they consider appropriate. The data is gently messy. It contains realistic inconsistencies in string normalization, multiple currency conventions for salary fields, a small fraction of missing values, and a time-based train/test split. These properties are intentional and are part of the assignment.
3. **Feature Engineering:** Design and implement at least five interaction features between candidate information and job information. Examples may include skill overlap, experience gap, salary mismatch, education compatibility, or location compatibility. Each engineered feature should be briefly explained and justified in the final report.
4. **Modeling:** You are building a scoring function $f(\text{job}, \text{candidate}) \rightarrow \text{relevance_score}$ using standard supervised learning. The final output of any model must be a continuous relevance score that can be used to rank candidates within each job_id. If a regression model is used, its predicted value can be used directly as the relevance score. If a classification model is used, participants should convert class probabilities into a single ranking score, for example by using the expected relevance. This score is then used only for sorting candidates within the same job_id. The evaluation metric directly measures ranking quality, not prediction accuracy. Standard machine learning models are completely acceptable. Ranking-oriented models such as LGBMRanker, XGBRanker, and CatBoostRanker may also be used, but they are not mandatory. The primary objective is to build an effective ranking or scoring pipeline. Implement an appropriate cross-validation strategy for the ranking task. A recommended approach

is to use GroupKFold(groups=job_id) in order to preserve the ranking context and ensure that samples belonging to the same job_id remain within the same fold. Participants are also encouraged to carefully consider data leakage, validation consistency, and fair evaluation methodology throughout the project.

5. **Evaluation Metrics:** Evaluation is done separately for each job. Candidates must only be compared with other candidates who applied to the same job_id, not with candidates from other jobs.

For each job, sort candidates by the predicted relevance score in descending order. The candidate with the highest predicted score gets rank 1. For each job, take the top 10 ranked candidates and compute:

$$DCG@10 = \sum_{i=1}^{10} \frac{2^{rel_i} - 1}{i + 1}$$

Here, rel_i is the true relevance label at position i. Higher relevance labels near the top receive more reward. If a job has fewer than 10 candidates, DCG@10 and IDCG@10 should be computed using all available candidates for that job. In other words, use the top min(10, number of candidates for that job) candidates.

Sort the same candidates by their true relevance labels in descending order. This creates the ideal ranking. Then compute DCG@10 for this perfect ranking.

$$IDCG@10 = DCG@10 \text{ of the ideal ranking}$$

Normalize the score for each job:

$$NDCG@10 = \frac{DCG@10}{IDCG@10}$$

A value closer to 1 means the predicted ranking is closer to the ideal ranking. If IDCG@10 is equal to zero for a job, meaning that none of the candidates for that job has a positive relevance label, NDCG@10 for that job will be defined as 0. This avoids division by zero and ensures that all jobs can be included consistently in the final average score.

Repeat the process for all jobs and average the results:

$$Score = \text{mean } NDCG@10 \text{ across all jobs}$$

For the secondary metric, the original graded relevance labels are first converted into binary labels. Candidates with labels 3 or 4 are considered relevant, while candidates with labels 0, 1, or 2 are considered non-relevant.

For each job, look at the top 5 ranked candidates. At each relevant position i, compute:

$$Precision@i = \text{number of relevant candidates in top } i / i$$

Only positions containing relevant candidates are used.

For each job, Average Precision@5, or AP@5, is computed by averaging Precision@i values only at positions where the candidate is relevant within the top 5 results.

$$AP@5 = (1 / \min(\text{number of relevant candidates}, 5)) \times \sum Precision@i \times rel_i$$

where rel_i is 1 if the candidate at position i is relevant and 0 otherwise. If a job has no relevant candidates, AP@5 for that job is defined as 0. Finally, average AP@5 across all jobs:

$MAP@5 = \text{mean } AP@5 \text{ across all jobs}$

NDCG@10 is the primary scoring metric, while MAP@5 is reported as a secondary metric for analyzing top-5 ranking quality.

Prediction & Submission:

- **Output:** Submit your predictions as a **CSV file**.
- **Format:** The CSV must contain exactly two columns:
 - application_id: The unique identifier for each test application instance.
 - score: Your model's predicted relevance score for that application. This score will be used to rank candidates within each job_id. Higher scores indicate candidates predicted to be more relevant for the job. (The score does not need to be a class label and does not need to be between 0 and 1. It only needs to provide a meaningful ordering of candidates within the same job posting.)

To participate in the competition related to this task Click on the competition link: [Quera Competition](#).

Good luck with the competition!